

MODULE 3

Bottom-Up Parsing:

Handle Pruning, Shift Reduce parsing, Operator precedence parsing (Concept only), LR parsing – Constructing SLR, LALR and Canonical LR Parsing tables.

3.1 BOTTOM-UP PARSING

A bottom-up parse starts with the string of terminals itself and builds from the leaves upward, working backwards to the start symbol by applying the productions in reverse.

Along the way, a bottom-up parser searches for substrings of the working string that match the right side of some production.

When it finds such a substring, it *reduces* it, i.e., substitutes the left side non-terminal for the matching right side. The goal is to reduce all the way up to the start symbol and report a successful parse.

3.1.1 SHIFT REDUCE PARSING

- ✚ Shift reduce parsing attempts to construct a parse tree for an input string beginning at the leaves (bottom) and working up towards the root (top).
- ✚ This can be considered as the process of “reducing” a string w to the start symbol of a grammar.
- ✚ At each reduction step parser searches for substrings of the working string that match the right side of some production.
- ✚ When it finds such a substring, it reduces it, i.e., substitutes the left side non-terminal for the matching right side. The goal is to reduce all the way up to the start symbol and report a successful parse.

Ex: Consider the grammar,

$$\begin{array}{ll} S \rightarrow aABb & B \rightarrow b \\ A \rightarrow aA \mid a & \text{Reduce the sentence 'aaabb' to } S. \end{array}$$

Soln:

input string: aaabb
 aaAbb
 aAbb
 aABb
 S

Handle:

- Informally, a **handle** of a string is a substring that matches the right side of a production

rule.

- But not every substring matches the right side of a production rule is handle.
- A **handle** of a right sentential form $\gamma (\equiv \omega\beta\alpha)$ is a production rule $A \rightarrow \beta$ and a position of γ , where the string β may be found and replaced by A to produce the previous right-sentential form in a rightmost derivation of γ . i.e.,

$$S \Rightarrow \alpha A \omega \Rightarrow \omega \beta \alpha$$

Handle Pruning:

A right-most derivation in reverse can be obtained by **handle-pruning**. That is we start with a string of terminals 'w' that we wish to parse.

Ex: Consider the grammar,

$$E \rightarrow E+T \mid T$$

$$T \rightarrow T^*F \mid F$$

$$F \rightarrow (E) \mid \text{id} \quad \text{Reduce the input string 'id+id*id' to start symbol E.}$$

Soln:

<u>Right-Most Sentential Form</u>	<u>Reducing Production</u>
<u>id</u> +id*id	$F \rightarrow \text{id}$
F+id*id	$T \rightarrow F$
<u>T</u> +id*id	$E \rightarrow T$
E+ <u>id</u> *id	$F \rightarrow \text{id}$
E+F*id	$T \rightarrow F$
E+T* <u>id</u>	$F \rightarrow \text{id}$
E+T*F	$T \rightarrow T^*F$
E+ <u>T</u>	$E \rightarrow E+T$
E	

Handles are underlined in the right-sentential forms.

A Stack Implementation of Shift-Reduce Parsing:

There are two problems in this technique.

1. To locate the substring to be reduced in a right-sentential form.
2. To determine what production to choose in case there is more than one production with that substring on the right side.

A convenient way to implement a shift-reduce parse is to use stack,

✚ A stack to hold grammar symbols and an input buffer to hold the string 'w'.

✚ As we shall see, the handle always appears at the top of the stack just before it is identified as the handle.

✚ We use \$ to mark the bottom of the stack and also the right end of the input. Initially, the stack is empty, and the string w is on the input, as follows:

STACK	INPUT
\$	w \$

✚ During a left-to-right scan of the input string, the parser shifts zero or more input symbols onto the stack, until it is ready to reduce a string β of grammar symbols on top of the stack.

- It then reduces β to the head of the appropriate production. The parser repeats this cycle until it has detected an error or until the stack contains the start symbol and the input is empty as follows:

STACK	INPUT
\$	S\$

- Upon entering this configuration, the parser halts and announces successful completion of parsing.

- There are actually four possible actions a shift-reduce parser can make:

- **Shift**
- **Reduce**
- **Accept**
- **Error**

- **Shift:** The next input symbol is shifted onto the top of the stack.
- **Reduce:** The parser knows the right end of the string to be reduced must be at the top of the stack. It must then locate the left end of the string within the stack and decide with what nonterminal to replace the string.
- **Accept:** Announce successful completion of parsing.
- **Error:** Discover a syntax error has occurred and calls an error recovery routine.

EXAMPLE 1:

Following figure steps through the actions a shift-reduce parser might take in parsing the input string $id_1 * id_2, id + id * id$ according to the expression grammar.

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

STACK	INPUT	ACTION
\$	$id_1 * id_2 \$$	shift
$\$ id_1$	$* id_2 \$$	reduce by $F \rightarrow id$
$\$ F$	$* id_2 \$$	reduce by $T \rightarrow F$
$\$ T$	$* id_2 \$$	shift
$\$ T *$	$id_2 \$$	shift
$\$ T * id_2$	$\$$	reduce by $F \rightarrow id$
$\$ T * F$	$\$$	reduce by $T \rightarrow T * F$
$\$ T$	$\$$	reduce by $E \rightarrow T$
$\$ E$	$\$$	accept

*Configurations of a shift-reduce parser on input $id_1 * id_2$*

<u>Stack</u>	<u>Input</u>	<u>Action</u>
\$	id+id*id\$	shift
\$id	+id*id\$	reduce by $F \rightarrow id$
\$F	+id*id\$	reduce by $T \rightarrow F$
\$T	+id*id\$	reduce by $E \rightarrow T$
\$E	+id*id\$	shift
\$E+	id*id\$	shift
\$E+id	*id\$	reduce by $F \rightarrow id$
\$E+F	*id\$	reduce by $T \rightarrow F$
\$E+T	*id\$	shift
\$E+T*	id\$	shift
\$E+T*id	\$	reduce by $F \rightarrow id$
\$E+T*F	\$	reduce by $T \rightarrow T*F$
\$E+T	\$	reduce by $E \rightarrow E+T$
\$E	\$	accept

3.1.2. OPERATOR PRECEDENCE PARSING

✚ Bottom-up parsers for a large class of context-free grammars can be easily developed using operator grammars.

✚ In an operator grammar, no production rule can have:

- at the right side (no production).
- two adjacent non-terminals at the right side.

✚ This property enables the implementation of efficient *operator-precedence parsers*.

EXAMPLE

$E \rightarrow AB$	$E \rightarrow EOE$	$E \rightarrow E+E \mid$
$A \rightarrow a$	$E \rightarrow id$	$E * E \mid$
$B \rightarrow b$	$O \rightarrow + \mid * \mid /$	$E / E \mid id$
not operator grammar	not operator grammar	operator grammar

Precedence Relations

✚ In operator-precedence parsing, we define three precedence relations between certain pairs of terminals as follows:

Relation	Meaning
$a < \cdot b$	a yields precedence to b (b has higher precedence than a)
$a = \cdot b$	a has the same precedence as b
$a \cdot > b$	a takes precedence over b (b has lower precedence than a)



The intention of the precedence relations is to find the handle of a right sentential form,
 $< \cdot$ with marking the left end,
 $\cdot$ appearing in the interior of the handle, and
 $\cdot >$ marking the right end.



In our input string $\$a_1a_2\dots a_n\$$, we insert the precedence relation between the pairs of terminals.



Example: Consider the string $\text{id} + \text{id} * \text{id}$ and the grammar is:

$$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid -E \mid \text{id}$$



The corresponding precedence relations is

	id	+	*	\$
Id		$\cdot >$	$\cdot >$	$\cdot >$
+	$< \cdot$	$\cdot >$	$< \cdot$	$\cdot >$
*	$< \cdot$	$\cdot >$	$\cdot >$	$\cdot >$
\$	$< \cdot$	$< \cdot$	$< \cdot$	$\cdot >$



Then the string with the precedence relations inserted is:

$$\$ < \cdot \text{id} \cdot > + < \cdot \text{id} \cdot > * < \cdot \text{id} \cdot > \$$$



$< \cdot$ is inserted between the leftmost $\$$ and id since $< \cdot$ is the entry in row $\$$ and column id .

Algorithm: Operator-Precedence parsing.

Input : An input string 'w' and a table of precedence relations.

Output : If 'w' is well formed, a skeletal parse tree, with a placeholder nonterminal 'E' then accept, otherwise, an error indication.

Method : Initially, the stack contains $\$$ and the input buffer the string $w\$$.

- Set ip to point to the first symbol of $w\$$;
- repeat forever
- if $\$$ is on top of the stack and ip points to $\$$ then
- return
- else begin
- let 'a' be the topmost terminal symbol on the stack and let 'b' be the symbol pointed to by 'ip';
- if $a < b$ or $a = b$ then begin
- push 'b' onto the stack;
- advance ip to the next input symbol;
- end;
- else if $a \cdot > b$ then /* reduce */
- repeat

- pop the stack
- until the top stack terminal is related by $<$ to the terminal most recently popped.
- else error();
- end

EXAMPLE:

Consider the grammar $E \rightarrow E + E \mid E * E \mid id$, reduce the input string 'id + id * id' to E using operator precedence parser

Soln:

<u>Stack</u>	<u>Input</u>	<u>Action</u>
\$	id+id*id\$	$a = \$, b = id; \$ < id$, push 'id'
\$id	+id*id\$	$id \cdot > +$, reduce $E \rightarrow id$
\$E	+id*id\$	$\$ < +$, push '+'
\$E+	id*id\$	$+ < id$, push 'id'
\$E+id	*id\$	$id \cdot > *$, reduce
\$E+E	*id\$	$+ < *$, push '*'
\$E+E*	id\$	$* < id$, push 'id'
\$E+E*id	\$	$id \cdot > \$$, reduce
\$E+E*E	\$	$* \cdot > \$$, reduce
\$E+E	\$	$+ \cdot > \$$, reduce
\$E	\$	accept

Algorithm: Constructing Precedence functions.

Input : An operator precedence matrix.

Output : Precedence functions representing the input matrix or an indication that none exist.

Method :

1. Create symbols f_a and g_a for each 'a' that is a terminal or '\$'.
2. Partition the created symbols into as many group as possible.
 - (i) If $a = b$ then f_a and g_b are in the same group.
 - (ii) If $a = b$ and $c = b$ then f_a and f_c must be in the same group.
3. Create a directed graph whose nodes are the groups found in (2)
 - (i) If $a < b$, place an edge from the group of g_b to the group of f_a .
 - (ii) If $a > b$, place an edge from the group of f_a to that of g_b .
4. If the constructed graph has a cycle, then no precedence functions exist.
5. If there are no cycles, let $f(a)$ be the length of the longest path beginning at the group of f_a . Let $g(a)$ be the length of the longest path from the group of g_a .

Example:

Consider the matrix

	id	+	*	\$
id	-	$\cdot >$	$\cdot >$	$\cdot >$
+	$< \cdot$	$\cdot >$	$< \cdot$	$\cdot >$
*	$< \cdot$	$\cdot >$	$\cdot >$	$\cdot >$
\$	$< \cdot$	$< \cdot$	$< \cdot$	-

Find the precedence functions.

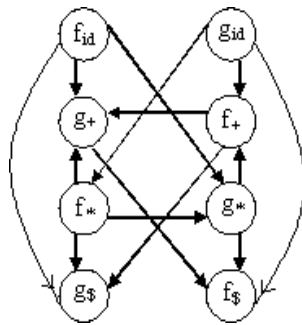
Soln:

Step 1:

Create symbols f_a and g_a for each f_{id} , g_{id} , f_+ , g_+ , f_* , g_* , $f_\$$ and $g_\$$.

Step 2:

Create a directed graph.



Step 3:

To find the longest path.

$$f_{id} = f_{id} \rightarrow g_* \rightarrow f_+ \rightarrow g_+ \rightarrow f_\$ \Rightarrow 4$$

$$g_{id} = g_{id} \rightarrow f_* \rightarrow g_* \rightarrow f_+ \rightarrow g_+ \rightarrow f_\$ \Rightarrow 5$$

$$f_+ = f_+ \rightarrow g_+ \rightarrow f_\$ \Rightarrow 2$$

$$g_+ = g_+ \rightarrow f_\$ \Rightarrow 1$$

$$f_* = f_* \rightarrow g_* \rightarrow f_+ \rightarrow g_+ \rightarrow f_\$ \Rightarrow 4$$

$$g_* = g_* \rightarrow f_+ \rightarrow g_+ \rightarrow f_\$ \Rightarrow 3$$

$$f_\$ = 0$$

$$g_\$ = 0$$

Step4:

Construct precedence functions table.

	+	*	id	\$
f	2	4	4	0
g	1	3	5	0

Advantages of operator-precedence parser:

1. Simplicity
2. Easy to implement
3. Operator precedence parsers have even built for entire language.

Disadvantages of operator-precedence parser:

1. It is hard to handle tokens like the minus sign, which has two different precedences.
2. Since the relationship between a grammar for the language being parsed and the operator precedence parser itself is tenuous.
3. Only a small class of grammar can be parsed.

Error Recovery in operator-precedence parsing:

An operator-precedence parser will detect an error, when,

1. No precedence relation holds between the terminal on top of the stack and the current input.
2. A handle has been found, but there is no production with this handle as a right side.

3.1.3. LR PARSING

LR parsing is most efficient method of bottom up parsing which can be used to parse large class of context free grammar.

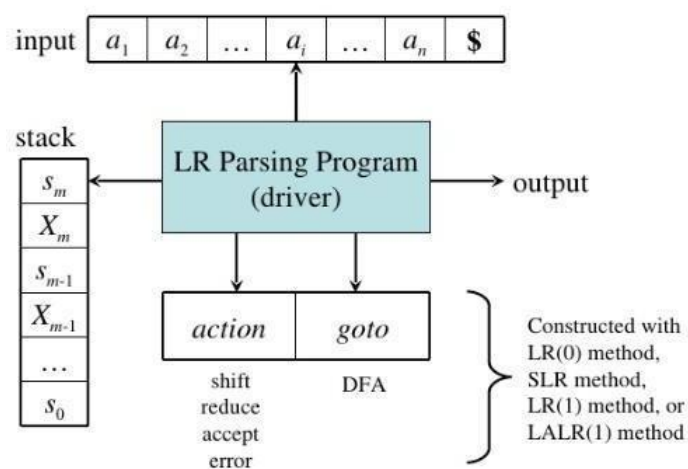
The technique is called LR(k) parsing; the “L” is for left to right scanning of input symbol, the “R” for constructing right most derivation in reverse, and the k for the number of input symbols of lookahead that are used in making parsing decision.

There are three types of LR parsing,

- SLR (Simple LR)
- CLR (Canonical LR)
- LALR (Lookahead LR)

The schematic form of LR parser is given below.

Model of an LR Parser



Requirements of LR parser:

- Input Buffer
- Stack
- Parsing Table
- LR Parsing Program

Input Buffer

The parsing program reads characters from an input buffer one at a time.

Stack

- ✚ Parsing program uses a stack to store string of the form $s_0X_1s_1X_2s_2X_3 \dots X_ms_m$ where s_m is on the top.
- ✚ X_i is a grammar symbol and S_i is the state
- ✚ Each state symbol summarizes the information contained in the stack below it
- ✚ Combination of state symbol on stack top and current input symbol are used to index the parsing table and determine the shift reduce parsing decision

Parsing Table

Consist of 2 parts

- action function
- goto function

Action

This function takes as arguments a state i and a terminal a (or $\$,$ the input end marker). The value of ACTION $[i, a]$ can have one of the four forms:

- i. Shift j , where j is a state.
- ii. Reduce by a grammar production $A \rightarrow \beta$.
- iii. Accept.
- iv. Error.

Goto

This function takes a state and grammar symbol as arguments and produces a state. If $GOTO[i, A] = I_j$, the GOTO also maps a state i and nonterminal A to state j .

Parsing Program works as follows

- ✚ It determines s_m , the state currently on top of stack and a_i the current input symbol
- ✚ It consults parsing action table entry for action $[s_m, a_i]$ which can have 4 values:
- ✚ The function goto takes a state and grammar symbol as arguments and produces a state.
 1. **Shift s , where s is a state**
 2. **Reduce by a grammar production $A \rightarrow b$**
 3. **Accept**
 4. **Error**
- ✚ The function goto takes a state and grammar symbol as arguments and produces a state.
- ✚ A configuration of LR parser is a pair whose first component is stack contents and second component is remaining input:

$$(s_0 X_1 s_1 X_2 \dots X_m s_m, a_i a_{i+1} \dots a_n \$)$$

- ✚ The next move of the parser is determined by reading a_i , the current input symbol and s_m , state on stack top and then consulting parsing action table entry action $[s_m, a_i]$.



The configuration resulting after each of four types of moves are as follows:

1. If action $[sm, ai] = \text{shift } s$, the parser executes a shift move, entering the configuration

(s0 X1 s1 X2 ... Xmsm, ai ai+1 ... an \$)

Here the parser has shifted both the current input symbol a_i and the next state s , which is given in action $[sm, ai]$, onto the stack; a_{i+1} becomes the current input symbol.

2. If action $[sm, ai] = \text{reduce } A \rightarrow \alpha$, then the parser executes a reduce move, entering the configuration

(s0 X1 s1 X2 s2 ... X_{m-r} sm-r A s, ai ai+1 ... an \$)

where $s = \text{goto}[sm-r, A]$ and r is the length of α , the right side of the production. Here the parser first popped $2r$ symbols off the stack (r state symbols and r grammar symbols), exposing state $sm-r$. The parser then pushed both A , the left side of the production, and s , the entry for $\text{goto}[sm-r, A]$, onto the stack. The current input symbol is not changed in a reduce move.

3. If action $[sm, ai] = \text{accept}$, parsing is completed.
4. If action $[sm, ai] = \text{error}$, the parser has discovered an error and calls an error recovery routine.

Algorithm for Constructing an SLR Parsing Table:

Input : An augmented grammar G' .

Output : The SLR parsing table functions action and goto for G' .

Method :

1. Construct $C = \{ I_0, I_1, \dots, I_n \}$, the collection of sets of LR(0) items for G' .
2. State ' i ' is constructed from I_i . The parsing actions for state ' i ' are determined as follows,
 1. If $[A \rightarrow \alpha.a\beta]$ is in I_i , and $\text{goto}(I_i, a) = I_j$ then set action $[i, a]$ to shift ' j ', ' a ' must be a terminal.
 2. If $[A \rightarrow \alpha.]$ is in I_i , then set action $[i, a]$ to "reduce $A \rightarrow \alpha$ " for all ' a ' in FOLLOW(A). Here A may not be S' .
 3. If $[S' \rightarrow S.]$ is in I_i , then set action $[i, \$]$ to "accept".
 If a conflict results from the above rules, the grammar is said not to be SLR(1).
3. If $\text{goto}(I_i, A) = I_j$, then $\text{goto}[i, A] = j$.
4. All entries not defined by rules(2) and (3) are made "error".
5. The initial state of the parser is the one constructed from the set of items containing $[S' \rightarrow .S]$.

Algorithm: LR Parsing algorithm (Stack Implementation).

Input : An input string ' w ' and an LR parsing table with functions action and goto for a grammar G .

Output : If ' w ' is in $L(G)$, a bottom-up parse for ' w '; otherwise, an error indication.

Method : Initially, the parser has S_0 on its stack, where S_0 is the initial state, and $w\$$ in the input buffer. Set ip to point to the first symbol of $w\$$;
repeat forever begin

Let S be the state on top of the stack and ' a ' the symbol

```

    pointed to by ip; if action[S, a] = shift S then begin
        Push 'a' then S on top of the stack, advance ip to the next input symbol
    end
    else if action[S, a] = reduce  $A \rightarrow \beta$ 
        then begin Pop  $2 * |\beta|$  symbols
            off the stack;
            Let S be the state now on top of the
            stack; Push A then goto[S', A] on top
            of the stack; Output the production
             $A \rightarrow \beta$ 
        end
    else if action[S, a] = accept then
        return
    else error()
end
end

```

Different types of LR Parsers

All LR parsers use the same parsing algorithm but the difference is only in terms of the construction of parsing tables. There are three widely used algorithms available for constructing an LR parsing Tables. Different types of LR parsers are:

SLR(1) - Simple LR

- Works on smallest class of grammar.
- Few number of states, hence very small table.
- Simple and fast construction.

LR(1) - LR parser

- Also called as Canonical LR parser.
- Works on complete set of LR(1) Grammar.
- Generates large table and large number of states.
- Slow construction.

LALR(1) - Look ahead LR parser

- Works on intermediate size of grammar.
- Number of states are same as in SLR(1).

Reasons for attractiveness of LR parser

-  LR parsers can handle a large class of context-free grammars.
-  The LR parsing method is a most general non-back tracking shift-reduce parsing method.
-  An LR parser can detect the syntax errors as soon as they can occur.
-  LR grammars can describe more languages than LL grammars.

Drawbacks of LR parsers

- ✚ It is too much work to construct LR parser by hand. It needs an automated parser generator.
- ✚ If the grammar contains ambiguities or other constructs then it is difficult to parse in a left-to-right scan of the input.

3.1.3.1 SIMPLE LR (SLR) PARSER:

LR(0) Item

- ✚ LR parser using SLR parsing table is called an SLR parser.
- ✚ A grammar for which an SLR parser can be constructed is an SLR grammar.
- ✚ An LR(0) item (item) of a grammar G is a production of G with a dot at the some position of the right side.

Ex: for the productions $A \rightarrow aBb$

Possible LR(0) Items are:

$A \rightarrow .aBb$

$A \rightarrow a.Bb$

$A \rightarrow aB.b$

$A \rightarrow aBb.$

- ✚ A production rule of the form $A \rightarrow \epsilon$ yields only one item $A \rightarrow .$
- ✚ Intuitively, an item shows how much of a production we have seen till the current point in the parsing procedure.
- ✚ A collection of sets of LR(0) items (the canonical LR(0) collection) is the basis for constructing SLR parsers.
- ✚ To construct the canonical LR(0) collection for a grammar we define:
 - **Augmented grammar**
 - **Two functions: CLOSURE and GOTO**

Augmented grammar

- ✚ G' is the augmented grammar of G with a new production rule $S' \rightarrow S$ where S' is the new starting symbol i.e,

$G' = G \cup \{S' \rightarrow S\}$ where S is the start state of G .

- ✚ The start state of $G' = S'$.
- ✚ This is done to signal to the parser when the parsing should stop to announce acceptance of input.

Kernel and Non-Kernel items

- ✚ Kernel items include the set of items that do not have the dot at leftmost end. $S' \rightarrow \cdot S$
- ✚ is an exception and is considered to be a kernel item.
- ✚ Non-kernel items are the items which have the dot at leftmost end.
- ✚ Sets of items are formed by taking the closure of a set of kernel items.

The Closure Operation

- ✚ If I is a set of LR(0) items for a grammar G , then $\text{closure}(I)$ is the set of LR(0) items constructed from I by the two rules:
 1. **Initially, every LR(0) item in I is added to $\text{closure}(I)$.**
 2. **If $A \rightarrow \alpha \cdot B\beta$ is in $\text{closure}(I)$ and $B \rightarrow \gamma$ is a production rule of G ; then $B \rightarrow \cdot \gamma$ will be in the $\text{closure}(I)$.**
- ✚ We will apply this rule until no more new LR(0) items can be added to $\text{closure}(I)$.

Computation of Closure

```

function closure ( I )
    begin
        J := I;
        repeat
            for each item  $A \rightarrow \alpha \cdot B\beta$  in J and each production
                 $B \rightarrow \gamma$  of G such that  $B \rightarrow \cdot \gamma$  is not in J do
                    add  $B \rightarrow \cdot \gamma$  to J
        until no more items can be added to J
    return J
end
  
```

The Closure Operation -- Example

$E' \rightarrow E$	$\text{closure}(\{E' \rightarrow \cdot E\}) =$
$E \rightarrow E+T$	$\{ E' \rightarrow \cdot E \} \leftarrow \text{kernel items}$
$E \rightarrow T$	$E \rightarrow \cdot E+T$
$T \rightarrow T * F$	$E \rightarrow \cdot T$
$T \rightarrow F$	$T \rightarrow \cdot T * F$
$F \rightarrow (E)$	$T \rightarrow \cdot F$
$F \rightarrow \text{id}$	$F \rightarrow \cdot (E)$
	$F \rightarrow \cdot \text{id} \}$

Goto Operation

✚ If I is a set of LR(0) items and X is a grammar symbol (terminal or non-terminal), then $\text{goto}(I, X)$ is defined as follows:

- If $A \rightarrow \alpha . X\beta$ in I then every item in $\text{closure}(\{A \rightarrow \alpha . X\beta\})$ will be in $\text{goto}(I, X)$.
- If I is the set of items that are valid for some viable prefix γ , then $\text{goto}(I, X)$ is the set of items that are valid for the viable prefix γX .

EXAMPLE

$I = \{ E' \rightarrow E., E \rightarrow E.+T \}$

$\text{goto}(I, +) = \{ E \rightarrow E+.T \}$

$T \rightarrow .T*F$

$T \rightarrow .F$

$F \rightarrow .(E)$

$F \rightarrow .id \}$

Construction of Canonical LR(0) Collection

✚ To create the SLR parsing tables for a grammar G , we will create the canonical LR(0) collection of the grammar G' .

ALGORITHM

Procedure items(G')

begin

$C := \{ \text{closure}(\{S' \rightarrow .S\}) \}$

repeat for each set of items I in C and each grammar symbol X if

$\text{goto}(I, X)$ is not empty and not in C

add $\text{goto}(I, X)$ to C

until no more set of LR(0) items can be added to C .

end

Ex.1: Consider the following grammar,

$E \rightarrow E + T \mid T$

$T \rightarrow T$

$* F \mid F$

$F \rightarrow$

$(E) \mid$

id

construct an SLR parser and parse the input string 'id*id+id'.

Soln:

- | | |
|---------------------------|-------------------------|
| (1) $E \rightarrow E + T$ | (4) $T \rightarrow F$ |
| (2) $E \rightarrow T$ | (5) $F \rightarrow (E)$ |
| (3) $T \rightarrow T * F$ | (6) $F \rightarrow id$ |

Step 1: To find Augmented Grammar:

- | | |
|-----------------------|---------------------|
| $E' \rightarrow E$ | |
| $E \rightarrow E + T$ | $T \rightarrow F$ |
| $E \rightarrow T$ | $F \rightarrow (E)$ |
| $T \rightarrow T * F$ | $F \rightarrow id$ |

Step 2: To find Closure of the items.

- | | |
|--------------------------|----------------------|
| $I_0: E' \rightarrow .E$ | $T \rightarrow .F$ |
| $E \rightarrow .E+T$ | $F \rightarrow .(E)$ |
| $E \rightarrow .T$ | $F \rightarrow .id$ |
| $T \rightarrow .T*F$ | |

Step 3: To find Goto(I, X).

- | | | | |
|--|--|---|--|
| $I_0:$ | $I_6: \text{goto}(I_1, +)$ | $I_5: \text{goto}(I_4, id)$ | $I_4: \text{goto}(I_7, ()$ |
| $E' \rightarrow .E$ | $E \rightarrow E+.T$ | $F \rightarrow id.$ | $F \rightarrow (.E)$ |
| $E \rightarrow .E+T$ | $T \rightarrow .T*F$ | | $E \rightarrow .E+T$ |
| $E \rightarrow .T$ | $T \rightarrow .F$ | $I_9: \text{goto}(I_6, T)$ | $E \rightarrow .T$ |
| $T \rightarrow .T*F$ | $F \rightarrow .(E)$ | $E \rightarrow E+T.$ | $T \rightarrow .T*F$ |
| $T \rightarrow .F$ | $F \rightarrow .id$ | $T \rightarrow T.*F$ | $T \rightarrow .F$ |
| $F \rightarrow .(E)$ | | | $F \rightarrow .(E)$ |
| $F \rightarrow .id$ | $I_7: \text{goto}(I_2, *)$ | $I_3: \text{goto}(I_6, F)$ | $F \rightarrow .id$ |
| | $T \rightarrow T*.F$ | $T \rightarrow F.$ | |
| $I_1: \text{goto}(I_0, E)$ | $F \rightarrow .(E)$ | | $I_{11}: \text{goto}(I_8,))$ |
| $E' \rightarrow E.$ | $F \rightarrow .id$ | $I_4: \text{goto}(I_6, ()$ | $F \rightarrow (E).$ |
| $E \rightarrow E.+T$ | | $F \rightarrow (.E)$ | |
| | $I_8: \text{goto}(I_4, E)$ | $E \rightarrow .E+T$ | $I_6: \text{goto}(I_8, +)$ |
| $I_2: \text{goto}(I_0, T)$ | $F \rightarrow (E.)$ | $E \rightarrow .T$ | $E \rightarrow E+.T$ |
| $E \rightarrow T.$ | $E \rightarrow E.+T$ | $T \rightarrow .T*F$ | $T \rightarrow .T*F$ |
| $T \rightarrow T.*F$ | | $T \rightarrow .F$ | $T \rightarrow .F$ |
| | $I_2: \text{goto}(I_4, T)$ | | |
| | $E \rightarrow T.$ | $F \rightarrow .(E)$ | $F \rightarrow .(E)$ |
| $I_3: \text{goto}(I_0, F)$ | $T \rightarrow T.*F$ | $F \rightarrow .id$ | $F \rightarrow .id$ |
| $T \rightarrow F.$ | | | |

I₄: goto(I₀, (
F → (.E)
E → .E+T
E → .T
T → .T*F
T → .F
F → .(E)
F → .id

I₅: goto(I₀, id)
F → id.

I₃: goto(I₄, F)
T → F.

I₄: goto(I₀, (
F → (.E)
E → .E+T
E → .T
T → .T*F
T → .F
F → .(E)
F → .id

I₅: goto(I₆, id)
F → id.

I₁₀: goto(I₇, F)
T → T*F.

I₅: goto(I₇, id)
F → id.

I₇: goto(I₉, *)
T → T*.F
F → .(E)
F → .id

Step 4: Construct SLR parsing table.

FOLLOW(E) = { \$, +,) }
 FOLLOW(T) = { \$, +,), * }
 FOLLOW(F) = { \$, +,), * }

state	action						goto		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

I₁: E' → E.	(1, \$)	====> ACCEPT
I₂: E → T.	(2, \$), (2,)), (2, +)	====> r2
I₃: T → F.	(3, \$), (3, +), (3,)), (3, *)	====> r4
I₅: F → id.	(5, *), (5, +), (5,)), (5, \$)	====> r6
I₉: E → E+T.	(9, \$), (9,)), (9, +)	====> r1
I₁₀: T → T*F.	(10, \$), (10, +), (10,)), (10, *)	====> r3
I₁₁: F → (E).	(11, *), (11, +), (11,)), (11, \$)	====> r5

Step 5: Stack Implementation.

<u>stack</u>	<u>input</u>	<u>action</u>
0	id*id+id\$	shift 5
0id5	*id+id\$	reduce by $F \rightarrow id$
0F3	*id+id\$	reduce by $T \rightarrow F$
0T2	*id+id\$	shift 7
0T2*7	id+id\$	shift 5
0T2*7id5	+id\$	reduce by $F \rightarrow id$
0T2*7F10	+id\$	reduce by $T \rightarrow T * F$
0T2	+id\$	reduce by $E \rightarrow T$
0E1	+id\$	shift 6
0E1+6	id\$	shift 5
0E1+6id5	\$	reduce by $F \rightarrow id$
0E1+6F3	\$	reduce by $T \rightarrow F$
0E1+6T9	\$	reduce by $E \rightarrow E + T$
0E1	\$	accept

EXAMPLE 2:

Construct SLR (1) GRAMMAR for

$S \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

Soln**AUGMENTED GRAMMAR**

$S' \rightarrow S$

$S \rightarrow E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

CANONICAL COLLECTION OF LR(0) ITEMS

I₀	I₃ = Go to (I₀, F)	I₇ = Go to (I₅, T)
$S' \rightarrow S$	$T \rightarrow F \bullet$	$E \rightarrow E + T \bullet$
$S \rightarrow \bullet E$		
$E \rightarrow \bullet E +$	I₄ = Go to (I₀, id)	I₃ = Go to (I₅, F)
$TE \rightarrow \bullet T$	$F \rightarrow id \bullet$	$T \rightarrow F \bullet$
$T \rightarrow \bullet T *$		
$FT \rightarrow \bullet F$	I₅ = Go to (I₁, +)	I₄ = Go to (I₅, id)
$F \rightarrow \bullet id$	$E \rightarrow E + \bullet T$	$F \rightarrow id \bullet$
	$T \rightarrow \bullet T * F$	
I₁ = Go to (I₀, E)	$T \rightarrow \bullet F$	I₈ = Go to (I₆, F)
$S' \rightarrow E \bullet$	$F \rightarrow \bullet id$	$T \rightarrow T * F \bullet$
$E \rightarrow E \bullet + T$		
	I₆ = Go to (I₂, *)	I₄ = Go to (I₆, id)
I₂ = Go to (I₀, T)	$T \rightarrow T \bullet *$	$F \rightarrow id \bullet$
$E \rightarrow T \bullet T$	$\bullet FF \rightarrow \bullet id$	
$T \bullet \rightarrow * F$		

NUMBER THE PRODUCTIONS

1. $S \rightarrow E$
2. $E \rightarrow E + T \mid T$
3. $T \rightarrow T * F \mid F$
4. $F \rightarrow id$

SLR(1) TABLE

FOLLOW (S) = {\$}

FOLLOW (E) = {+, \$}

FOLLOW (T) = {*, +, \$}

FOLLOW (F) = {*, +, \$}

- I1 contains the final item which drives $S \rightarrow E \bullet$ and follow (S) = { \$ }, so action {I1, \$} = Accept
- I2 contains the final item which drives $E \rightarrow T \bullet$ and follow (E) = { +, \$ }, so action {I2, +} = R2, action {I2, \$} = R2
- I3 contains the final item which drives $T \rightarrow F \bullet$ and follow (T) = { +, *, \$ }, soaction {I3, +} = R4, action {I3, *} = R4, action {I3, \$} = R4
- I4 contains the final item which drives $F \rightarrow id \bullet$ and follow (F) = { +, *, \$ }, soaction {I4, +} = R5, action {I4, *} = R5, action {I4, \$} = R5
- I7 contains the final item which drives $E \rightarrow E + T \bullet$ and follow (E) = { +, \$ }, soaction {I7, +} = R1, action {I7, \$} = R1
- I8 contains the final item which drives $T \rightarrow T * F \bullet$ and follow (T) = { +, *, \$ }, soaction {I8, +} = R3, action {I8, *} = R3, action {I8, \$} = R3.

States	Action				Go to		
	id	+	*	\$	E	T	F
I ₀	S ₄				1	2	3
I ₁		S ₅		Accept			
I ₂		R ₂	S ₆	R ₂			
I ₃		R ₄	R ₄	R ₄			
I ₄		R ₅	R ₅	R ₅			
I ₅	S ₄					7	3
I ₆	S ₄						8
I ₇		R ₁	S ₆	R ₁			
I ₈		R ₃	R ₃	R ₃			

EXAMPLE 3:

Consider the following grammar,

$$\begin{aligned}
 S &\rightarrow L = R \\
 S &\rightarrow R \\
 L &\rightarrow * R \\
 L &\rightarrow id \\
 R &\rightarrow L
 \end{aligned}$$

Check whether the grammar is SLR(1)?

Soln:

AUGMENTED GRAMMAR

$$\begin{aligned}
 S' &\rightarrow S \\
 S &\rightarrow L = R \\
 S &\rightarrow R
 \end{aligned}$$

$$L \rightarrow * R$$

$$L \rightarrow id$$

$$R \rightarrow L$$

CANONICAL COLLECTION OF LR(0) ITEMS

I0	I5= Go to (I0, id)	I9= Go to (I6 , R)
$S' \rightarrow \bullet S$	$L \rightarrow id \bullet$	$S \rightarrow L = R \bullet$
$S \rightarrow \bullet L = R$		
$S \rightarrow \bullet R$	I6= Go to (I2 , =)	I8= Go to (I6 , L)
$L \rightarrow \bullet * R$	$S \rightarrow L = \bullet R$	$R \rightarrow L \bullet$
$L \rightarrow \bullet id$	$R \rightarrow \bullet L$	
$R \rightarrow \bullet L$	$L \rightarrow \bullet * R$	I7= Go to (I6 , L)
	$L \rightarrow \bullet id$	$L \rightarrow * R \bullet$
I1= Go to (I0, S)		
$S' \rightarrow S \bullet$	I7= Go to (I4 , R)	I4= Go to (I6 , *)
	$L \rightarrow * R \bullet$	$L \rightarrow * \bullet R$
I2= Go to (I0, L)		$R \rightarrow \bullet L$
$S \rightarrow L \bullet = R$	I8= Go to (I4 , L)	$L \rightarrow \bullet * R$
$R \rightarrow L \bullet$	$R \rightarrow L \bullet$	$L \rightarrow \bullet id$
I3= Go to (I0, R)	I4= Go to (I4 , *)	I5= Go to (I6 , id)
$S \rightarrow R \bullet$	$L \rightarrow * \bullet R$	$L \rightarrow id \bullet$
	$R \rightarrow \bullet L$	
I4= Go to (I0, *)		
$L \rightarrow * \bullet R$	$L \rightarrow \bullet * R$	
$R \rightarrow \bullet L$	$L \rightarrow \bullet id$	
$L \rightarrow \bullet * R$	I5= Go to (I4 , id)	
$L \rightarrow \bullet id$	$L \rightarrow id \bullet$	

NUMBER THE PRODUCTIONS

1. $S \rightarrow L = R$
2. $S \rightarrow R$
3. $L \rightarrow * R$
4. $L \rightarrow id$
5. $R \rightarrow L$

PARSING TABLEI₁ = S' → S •I₂ = R → L •I₃ = S → R •I₅ = L → id •I₇ = L → * R •I₈ = R → L •I₉ = S → L = R •

FOLLOW (S) = {\$}

FOLLOW (L) = {=, \$}

FOLLOW (R) = {=, \$}

(2, =), (2, \$) =====> r5

(3, \$) =====> r2

(5, =), (5, \$) =====> r4

(7, =), (7, \$) =====> r3

(8, \$), (8, =) =====> r5

(9, \$) =====> r1

STATE	ACTION				GOTO		
	id	*	=	\$	R	L	S
0	s5	s4			3	2	1
1	ACCEPT						
2			s6/ r5	r5			
3			r2				
4	s5	s4			7	8	
5			r4	r4			
6	s5	s4			9	8/7	
7			r3	r3			
8			r5	r5			
9				r1			

Here there is a shift reduce conflict in (2,=) so grammar is not SLR(1)

3.1.3.2. CANONICAL LR(CLR)PARSER

- In SLR method, the state i makes a reduction by $A \rightarrow \alpha$ when the current token is a :
 - if the $A \rightarrow \alpha$ is in the I_i and a is $\text{FOLLOW}(A)$
 - In some situations, when state i appears on stack top, the viable prefix $\beta\alpha$ on the stack is such that βA cannot be followed by a in right sentential form.
- Thus the reduction by $A \rightarrow \alpha$ would be invalid on input a .
- Because of that we go for : Canonical LR Parser
 - In this, it is possible to carry more information in the state that will allow us to avoid some of these invalid reductions
- To avoid some of invalid reductions, the states need to carry more information.
- Extra information is put into a state by including a terminal symbol as a second component in an item.
- A LR(1) item is:

$$A \rightarrow \alpha.\beta,a \quad \text{where } a \text{ is the look-head of the LR(1) item}$$

(a is a terminal or end-marker.)

- Such an object is called LR(1) item.
 - 1 refers to the length of the second component
 - The lookahead has no effect in an item of the form $[A \rightarrow \alpha.\beta,a]$, where β is not ϵ .
 - But an item of the form $[A \rightarrow \alpha.,a]$ calls for a reduction by $A \rightarrow \alpha$ only if the next input symbol is a .
- When β (in the LR(1) item $A \rightarrow \alpha.\beta,a$) is not empty, the look-head does not have any affect.
- When β is empty ($A \rightarrow \alpha.,a$), we do the reduction by $A \rightarrow \alpha$ only if the next input symbol is a (not for any terminal in $\text{FOLLOW}(A)$).

Construction of CLR (canonical LR) Parsing Tables Algorithm:

Construction of canonical- LR parsing tables.

INPUT: An augmented grammar G' .

OUTPUT: the canonical-LR parsing table functions ACTION AND GOTO for G' .**METHOD:**

1. Construct $C' = \{ I_0, I_1, \dots, I_n \}$, the collection of sets of LR(1) items for G' .
2. State i of the parser is constructed from I_i . The parsing action for state i is determined as follows.

- (a) If $[A \rightarrow \alpha.a\beta, b]$ is in I_i , and $\text{GOTO}(I_i, a) = I_j$, then set $\text{ACTION}[i, a]$ to "shift j ". Here a must be a terminal.
- (b) If $[A \rightarrow \alpha., a]$ is in I_i , $A \neq S'$, then set $\text{ACTION}[i, a]$ to "reduce $A \rightarrow \alpha.$ ".
- (c) If $[S' \rightarrow S., \$]$ is in I_i , then set $\text{ACTION}[i, \$]$ to "accept".

If any conflicting actions result from the above rules, we say the grammar is not LR(1). The algorithm fails to produce a parser in this case.

3. The goto transitions for state i are constructed for all non terminals A using the rule: if $\text{GOTO}(I_i, A) = I_j$, then $\text{GOTO}[i, A] = j$.
4. All entries not defined by rules (2) and (3) are made "error".
5. The initial state of the parser is the one constructed from the set of items containing $[S' \rightarrow .S, \$]$.

EXAMPLE 1:

Construct CLR parsing table for the following grammar

$S \rightarrow CC$
 $C \rightarrow cC \mid d$

Soln:

AUGMENTED GRAMMAR

$S' \rightarrow S$

$S \rightarrow CC$
 $C \rightarrow cC$
 $C \rightarrow d$

CANONICAL COLLECTION OF LR(1) ITEMS

I_0 $S' \rightarrow \cdot \underline{S}, \$$ $S \rightarrow \cdot CC, \$$ $C \rightarrow \cdot \underline{cC}, c \mid d$ $C \rightarrow \cdot d, c \mid d$	$I_6 = \text{GOTO}(I_2, c)$ $C \rightarrow c \cdot \underline{C}, \$$ $C \rightarrow \cdot \underline{cC}, \$$ $C \rightarrow \cdot d, \$$	$I_8 = \text{GOTO}(I_3, C)$ $C \rightarrow c \cdot \underline{C}, c \mid d$
$I_1 = \text{GOTO}(I_0, S)$ $S' \rightarrow S \cdot, \$$	$I_7 = \text{GOTO}(I_2, d)$ $C \rightarrow d \cdot, \$$	$I_3 = \text{GOTO}(I_3, c)$ $C \rightarrow c \cdot \underline{C}, c \mid d$ $C \rightarrow \cdot \underline{cC}, c \mid d$ $C \rightarrow \cdot d, c \mid d$
$I_2 = \text{GOTO}(I_0, C)$ $S \rightarrow C \cdot C, \$$ $C \rightarrow \cdot \underline{cC}, \$$ $C \rightarrow \cdot d, \$$	$I_4 = \text{GOTO}(I_0, d)$ $C \rightarrow d \cdot, c \mid d$	$I_4 = \text{GOTO}(I_3, d)$ $C \rightarrow d \cdot, c \mid d$
	$I_5 = \text{GOTO}(I_2, C)$ $S \rightarrow CC \cdot, \$$	$I_9 = \text{GOTO}(I_6, C)$ $C \rightarrow c \cdot \underline{C}, \$$

I₃ = GOTO (I₀, c) $C \rightarrow c \bullet C, c \mid d$ $C \rightarrow \bullet c C, c \mid d$ $C \rightarrow \bullet d, c \mid d$ **I₆ = GOTO (I₂, c)** $C \rightarrow c \bullet C, \$$ $C \rightarrow \bullet c C, \$$ $C \rightarrow \bullet d, \$$ **I₆ = GOTO (I₆, c)** $C \rightarrow c \bullet C, \$$ $C \rightarrow \bullet c C, \$$ $C \rightarrow \bullet d, \$$ **I₄ = GOTO (I₀, d)** $C \rightarrow d \bullet, c \mid d$ **I₇ = GOTO (I₂, d)** $C \rightarrow d \bullet, \$$ **I₇ = GOTO (I₆, d)** $C \rightarrow d \bullet, \$$ **I₅ = GOTO (I₂, C)** $S \rightarrow CC \bullet, \$$ **NUMBER THE PRODUCTIONS**

1. $S \rightarrow CC$
2. $C \rightarrow cC$
3. $C \rightarrow d$

PARSING TABLE**I₁ = S' → S•, \$****I₄ = C → d•, c | d****I₅ = S → CC•, \$****I₇ = C → d•, \$****I₈ = C → c C•, c | d****I₉ = C → c C•, \$**

STATE	ACTION			GOTO	
	c	d	\$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

3.1.3.3. LOOK AHEAD LR PARSER(LALR)

- LALR stands for Look ahead LR.
- It is intermediate in power and cost between the other two.
- The LALR method will work on most programming language grammars.
- The LALR table constructed is smaller than CLR tables.

Ex:

Construct LALR parser for the following grammar,

 $S \rightarrow CC$ $C \rightarrow cC$ $C \rightarrow d$ **Soln.**Consider the set I₃ and I₆. The set I₃ differs from I₆ only in second component, merge

this sets and call it as I_{36} .

$I_{36}: C \rightarrow c.C, c/d/\$$

$C \rightarrow .cC, c/d/\$$

$C \rightarrow .d, c/d/\$$

I_4 and I_7 are replaced by

$I_{47}: C \rightarrow d., c/d/\$$

I_8 and I_9 are replaced by

$I_{89}: C \rightarrow cC., c/d/\$$

LALR Parsing table:

state	action			goto	
	c	d	\$	S	C
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			8
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		9
